Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Vize duyurusu ve dönem içi değerlendirme oranları.
- Metin işleme algoritmaları (text processing algorithms):
  - Ağaçların bağlı (linkli) gösterimi.
  - Trie ağaçları, sıkıştırılmış trie ve trie'nin ikili ağaç (binary tree) olarak gösterimi.
  - Sonek ağacı (suffix tree).
  - En uzun ortak alt dizi (longest common subsequence, LCS) ve dinamik programlama (dynamic programming) çözümü.
  - En uzun ortak alt dize (longest common substring).
  - Minimum düzenleme uzaklığı (minimum edit distance) ve Levenshtein uzaklığı.
- Ara sonrası: yazım hatası düzeltme üzerine soru-cevap ve bir öğrencinin adres verisi projesi.
- Bilgiye erişim sistemleri (information retrieval, IR):
  - Kısa tarihçe ve IR sistemi mimarisi.
  - Web tarayıcı (spider), indeksleme ve sayfa önemi.
  - IR modellerinin sınıflandırılması.
  - Standart Boole modeli (standard Boolean model).
  - Genişletilmiş Boole modeli (extended Boolean model) ve p-normu.
  - Vektör uzayı modeli (vector space model), TF-IDF, kosinüs benzerliği (cosine similarity) ve iç çarpım (inner product).
  - Ters çevrilmiş dosya indeksi (inverted file index) ve tam ters çevrilmiş indeks (full inverted index).
  - Bulanık (fuzzy) mantık ve bulanık bilgi erişimi, karma min-maks modeli (mixed min and max model, MMM).

Hocanın Özellikle Vurguladığı Kısımlar

- Vize 17 Aralık 2020'de yapılacak. Test usulü olacak:
  - Yaklaşık 30 soru, süre 2 saat.
  - Bütün konular dahil.
  - Bazı sorular doğrudan okuyup cevaplanır. Bazılarında kağıtta çözüm yapıp sonucu şıklarda aramak gerekir.
  - Defter ve kitap açık olacak; hoca bunu zaten bildiğini söylüyor.
- Bir sonraki hafta (10 Aralık) ders anlatılacak, ardından vize gelecek.
- Yeni yönetmeliğe göre en az iki değerlendirme gerekiyor. Yıl içinin %60'ı, yıl sonunun %40'ı alınacak.
- Hocanın beklentisi: derste görülen her şeyi ezberlemek değil, ihtiyaç anında "böyle bir yöntem görmüştük" diyebilmek.
- Doktora öğrencilerine not: yeterlilik sınavında hoca derste vurguladığı konulardan soru sorar.
- Kelime temsili (word representation), word2vec ve BERT gibi konular hocanın kullandığı ders kitaplarında yok. Bunlar son 5 yılın konuları. İnternette tutorial ve ders slaytı olarak kolayca bulunur.
- Trie'de arama maliyeti, aranan kelimenin uzunluğu kadardır. Ortak önekler tekrar saklanmaz, hafıza verimli kullanılır.
- Edit distance'ta yol (hangi işlemlerin yapıldığı) kişiden kişiye değişebilir. Uzaklık değeri

ise değişmez.
- Standart Boole modelinin en büyük eksiği sıralama (ranking) yapmamasıdır. Genişletilmiş Boole, bulanık ve vektör uzayı modelleri bu eksiği gidermek için çıkmıştır.
- Ağırlıklandırma (weighting) formülü serbesttir. Probleminizde başka bir formül daha iyi sonuç veriyorsa onu kullanabilirsiniz.
- Bulanık mantıkta değerler olasılık değil üyelik derecesidir (degree of membership). Toplamları 1'i geçebilir.
- Bir sonraki konu cebirsel modellerden LSA (gizli anlamsal analiz, latent semantic analysis). Hoca bu konuyu yeterlilik sınavında sorduğunu, cevap alamadığını ve bu yüzden daha iyi anlatacağını söylüyor.

Kısa Tekrar Notları

- Trie adı "retrieval" kelimesinden gelir.
- Trie'de kök boş dizedir. Kökten yaprağa giden her yol bir kelime verir.
- Türkçede kökten en fazla 28 dal çıkar: alfabede 29 harf var ama "ğ" ile başlayan kelime yok.
- Trie'nin derinliği en uzun kelimenin uzunluğu kadardır. Cümle saklanıyorsa en uzun cümlenin kelime sayısı kadardır.
- Sıkıştırılmış trie: tek çocuklu düğüm zincirleri tek bir kenarda birleştirilir.
- Sonek ağacı: dizenin bütün sonekleri ağaca eklenir. Dize sonu $ ile işaretlenir.
- Sonek ağacı alt dize arama için kullanılır. Kullanım alanları: biyoinformatik (DNA dizileri), arama motorları, veritabanları, derleyiciler.
- LCS'de alt dizinin elemanları ardışık olmak zorunda değil. Yalnızca sıra bozulmamalı.
- LCS ve edit distance tablo tabanlı dinamik programlama ile çözülür. Zaman ve yer karmaşıklığı $O(mn)$.
- Edit distance işlemleri: ekleme (insertion), silme (deletion), yerine koyma (substitution).
- Levenshtein uzaklığında ekleme ve silme 1, yerine koyma 2 maliyetlidir.
- "intention" ile "execution" arasındaki uzaklık: tüm işlemler 1 maliyetliyse 5, Levenshtein ile 8.
- Standart Boole modeli: AND kesişim (intersection), OR birleşim (union). Sonuçlar sırasızdır.
- Genişletilmiş Boole modeli: terim ağırlıklarıyla benzerlik hesaplar, sonuçları sıralar.
- Kosinüs benzerliği 1'e ne kadar yakınsa belgeler o kadar benzer.
- Ters çevrilmiş indeks: kelime → belge listesi. Tam ters çevrilmiş indeks: kelime → (belge, konum) listesi.
- Bulanık işlemler: OR = maksimum, AND = minimum, NOT (tümleyen) = $1 - x$.
- MMM modeli: hem max hem min değerini katsayılarla birleştirir.

Detaylı Açıklamalar

Metin işleme algoritmalarının amacı

- Hoca bu bölümde "kelime" ya da "metin" demesinin önemli olmadığını söylüyor. İşlem kelime, cümle veya belge (document) düzeyinde yapılabilir.
- Cevaplanan sorular:
  - Kelimeleri hangi veri yapısında saklamalıyım ki hızlı erişeyim ve tekrar eden kalıplar için boşuna yer harcamayayım?
  - Uzun dizeler içinde en uzun eşleşmeyi nasıl bulurum?
  - Yanlış yazılmış bir kelime ile doğrusu arasındaki uzaklığı nasıl ölçerim? Örnek: "kabak" yerine yanlışlıkla "tabak" yazılması.

Ağaçların bağlı gösterimi

- Her düğüm bellekte üç gözle tutulur:
  - Sol göz: düğümdeki bilgi.
  - Orta göz: düğümün nereden geldiği (ebeveyn bağlantısı). Kökte boştur.
  - Sağ göz: düğümden gidilebilecek çocuklar.
- Örnekte kök B'nin 3 çocuğu var: A, D, F. D'nin iki çocuğu var: C ve E.
- Adresler izlenerek kimin kimin altında olduğu kolayca bulunur.

Trie ağaçları

- Adı “retrieval” kelimesindeki harflerden gelir.
- Bir metin, cümle ya da sözlük saklanabilir. Örnek: TDK sözlüğündeki kelimeler (anlamları olmadan).
- Kurallar:
  - Kök düğüm boş dizeyle başlar.
  - Her düğümden bir harfi taşıyan yalnızca bir dal çıkabilir.
  - Dallarda ilerledikçe harfler birleşir. Yaprağa gelindiğinde bir kelime elde edilir.
  - Her yaprağa giden tek bir yol vardır.
- Avantaj: arama maliyeti aranan metnin boyu kadardır. Hafıza verimli kullanılır.
- Slayttaki örnek dizeler: AEEF, AD, BBFE, BBFG, C.
  - AEEF ve AD aynı “A” dalını paylaşır. Trie, aynı önek için ikinci bir dal açılmasını engeller.
- Sıkıştırılmış trie: E→E→F zincirinde başka dallanma olmadığı için tek kenar “EEF” yazılır. Aynı şekilde “BBF” tek kenar olur.
- Trie veritabanlarında kayıtları yerleştirmek için de kullanılabilir.
- Dizi tabanlı gösterim: her düğümde alfabedeki harfler kadar göz vardır. Örnek alfabe A, E, I, P, R. “RAP” için R gözünden A’ya, oradan P’ye inilir. Her gözde adres saklandığı için geriye doğru köke dönülebilir.

Trie’nin ikili ağaç olarak gösterimi

- Her düğümden en fazla iki dal çıkar:
  - Sağ bağlantı aynı seviyedeki bir sonraki harfi (kardeşi) gösterir.
  - Sol bağlantı bir alt seviyeye, yani kelimenin bir sonraki harfine iner.
- Arama: aynı seviyede sağa doğru aranan harf bulunana kadar ilerlenir. Bulununca sola, bir alt seviyeye inilir. Bu, özyinelemeli (recursive) bir işlem gibi çalışır.

Örnek: “ab”, “abla”, “aba”

```
(kök)
 └─ a
     └─ b            -> "ab"
         ├─ l
         │   └─ a    -> "abla"
         └─ a        -> "aba"
```

- Yeni bir kelime, önceden açılmış dalı kullanıyorsa ağaç o noktada yenilenir ve yeni dal eklenir.

Sonek ağacı (suffix tree)

- “Suffix” son ek demektir. Trie’ye benzer ama dizeler değil, bir dizenin sonekleri eklenir.
- Kullanım alanları:
  - Biyolojik veriler: DNA dizisinde bozulmanın nerede olduğunu aramak.
  - Arama motorları: “okul” aranınca “okullar”, “okullardan” gibi türetilmiş kelimelerin geçtiği sayfaları da getirmek.
  - Veritabanları ve derleyiciler (compiler).
- Asıl amaç alt dize (substring) bulmak. Örnek: “ABCADAC” içinde “BCA” kalıbı var mı?
- Dizenin sonuna $ konur. Bu, dizenin bittiğini gösterir.
- “ABAB"$dizesininsonekleri$:` $ABAB$,BAB`,` $AB$,B`,`‘.
- Sıkıştırılmış sonek ağacı:

```
(kök)
 ├─ AB ─┬─ AB$    -> ABAB$
 │      └─ $      -> AB$
 ├─ B ──┬─ AB$    -> BAB$
 │      └─ $      -> B$
 └─ $             -> $
```

- Ekleme mantığı: daha önce açılmış bir dal varsa yeniden kullanılır. Ayrım noktasında yeni düğüm açılır.
- Trie'den farkı: trie'de ayrı ayrı dizeler eklenir. Sonek ağacında tek dizenin sonekleri sırayla eklenir.
- Derste ikinci bir örnek tahtada çizildi ama karıştı. Hoca örneği birkaç kez baştan aldı. Bu kısım altyazıdan izlenebilir değil.

En uzun ortak alt dizi (LCS)

- Alt dizi (subsequence) için karakterlerin ardışık olması gerekmez. Yalnızca sıra korunmalı.
- "ACTTGCG" dizesi için örnekler:
  - "ACT", "ATTC", "T", "ACTTGC" alt dizidir.
  - "TTA" alt dizi değildir. A, iki T'den önce geldiği için sıra bozulur.
- Slaytta iki DNA dizisi için LCS uzunluğu 13 bulunuyor.
- Uygulama örneği: birden çok adı olan biri, formlara farklı kombinasyonlar yazmış olabilir ("Ali Hasan Yüksek", "Hasan Hüseyin Yüksek", "Ali Yüksek"). Bunların aynı kişi olduğunu bulmak için ortak alt dizi kullanılabilir. Hoca, aynı kişi kararının başka anahtar kelimelerle de desteklenmesi gerektiğini ekliyor.

Algoritma

- X'in boyu m, Y'nin boyu n. c uzunluk tablosu, b yön (ok) tablosu.
- 0. satır ve 0. sütun 0 ile doldurulur.
- Doldurma kuralı:

$$c[i,j] = \begin{cases} 0 & i = 0 \text{ ya da } j = 0 \\ c[i-1, j-1] + 1 & x_i = y_j \quad \text{(çapraz ok)} \\ \max(c[i-1, j],\ c[i, j-1]) & x_i \neq y_j \end{cases}$$

- Karakterler farklıysa: üstteki değer soldakinden büyük ya da eşitse üstteki yazılır ve yukarı ok konur. Değilse soldaki yazılır ve sol ok konur.
- Tablonun sağ alt köşesindeki değer LCS'nin uzunluğudur.
- Geri izleme: sağ alt köşeden başlanır. Karakterler eşitse karakter yazılır ve çapraza gidilir. Değilse ok yönünde gidilir. Bulunan karakterler ters sırada okunur.
- Hoca, tablonun sol üstten başlamak zorunda olmadığını, tersten de doldurulabileceğini söylüyor.

Örnek: X = ACTTGCG ($m = 7$), Y = ATTCGG ($n = 6$)

		A	T	T	C	G	G
	0	0	0	0	0	0	0
A	0	1↖	1←	1←	1←	1←	1←
C	0	1↑	1↑	1↑	2↖	2←	2←
T	0	1↑	2↖	2↖	2↑	2↑	2↑
T	0	1↑	2↖	3↖	3←	3←	3←
G	0	1↑	2↑	3↑	3↑	4↖	4↖
C	0	1↑	2↑	3↑	4↖	4↑	4↑
G	0	1↑	2↑	3↑	4↑	5↖	5↖

- Sağ alt köşe 5: LCS uzunluğu 5.
- Geri izleme sırasıyla G, G, T, T, A bulunur. Ters çevrilince LCS = "ATTGG".
- Hoca: algoritma basit ama uzun dizelerde elle en uzun eşleşmeyi bulmak neredeyse imkansız.

En uzun ortak alt dize (longest common substring)

- İki dize aynı sonek ağacına yerleştirilir. Slaytta bir dizenin dalları siyah, diğerininki kırmızı.
- Her iki rengin de geçtiği dallar ortak alt dizelerdir.

- Dinamik programlama kodu da slaytta var. Hoca ayrıntısına girmedi.

Minimum düzenleme uzaklığı (minimum edit distance)

- İki dize arasındaki farkı sayısal olarak ölçer. Hoca "benzerlik" değil "uzaklık" kelimesini vurguluyor.
- Örnek: "aba" ile "abla". "abla" yerine "alya" yazmak için B ile L'nin yerine koyulması ya da silme ve ekleme yapılması gibi yollar var.
- Kullanım alanları: dize karşılaştırma, yazım denetimi (spell checking), biyoinformatik, makine çevirisi (machine translation).
- İşlemler:
  - Silme (deletion).
  - Ekleme (insertion).
  - Yerine koyma (substitution): aslında bir silme ve bir ekleme.
- Maliyet:
  - Bazı yöntemlerde her işlem 1 maliyetlidir.
  - Levenshtein uzaklığında yerine koyma 2 maliyetlidir. Genelde bu kullanılır.

"intention" → "execution" hizalaması

```
I N T E _ N T I O N
_ E X E C U T I O N
d s s   i s
```

- _: boşluk (karşılığı yok). d: silme, s: yerine koyma, i: ekleme.
- Tüm işlemler 1 maliyetliyse: 1 + 3 + 1 = 5.
- Levenshtein ile: 3 yerine koyma × 2 = 6, + 1 ekleme + 1 silme = 8.

Algoritma

- $D(i, 0) = i$, $D(0, j) = j$.

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2 & x_i \neq y_j \\ 0 & x_i = y_j \end{cases} \end{cases}$$

- Çapraz geçiş yerine koymadır. Satır ve sütun boyunca geçişler silme ve eklemedir.
- Her göze, değerin nereden geldiğini gösteren ok konur. Birden çok yönden aynı minimum geliyorsa birden çok ok konur.
- Üç oklu bir gözde yerine koyma, silme veya ekleme seçilebilir. Hangisi seçilirse seçilsin toplam uzaklık değişmez.
- Hoca yerine koymayı seçmenin daha az adımla ilerlediğini söylüyor. Silme veya ekleme seçilirse yol daha uzun görünür ama toplam yine 8 olur.
- Matrisin sonundaki değer: 8.
- Karmaşıklık: zaman ve yer $O(mn)$. Hoca "n kare karmaşıklığında" diyor.

Ara sonrası soru-cevap: yazım hatası düzeltme

- Bir öğrenci, kelime düzeltme için hazır bir algoritma olup olmadığını sordu.
- Hocanın kendi öğrencisiyle yaptığı çalışma:
  - Yanlış yazılmış "cocuk" gibi bir kelimenin tüm c/ç kombinasyonları üretildi.
  - Her aday, cümledeki sağ ve sol komşu kelimelerle arama motoruna gönderildi.
  - En çok sonuç dönen aday doğru kabul edildi. Yaklaşık %90 başarı alındı.
- Başka yollar:
  - Sonlu durum makineleri (finite state machines).
  - n-gram modelleriyle hangi karakterden sonra hangisinin gelme olasılığını çıkarmak.
  - Saklı Markov modelleri (Hidden Markov Models, HMM).
- Türkçeye özgü hazır bir algoritma yok. Problem Türkçeye özgü olduğu için kendi verinizle çözmelisiniz.

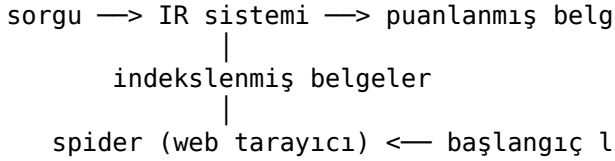
- Bağlam (context) şart: “kıs” ile “kış”, “kus” ile “küs” gibi iki doğru kelime karışabilir. Kelime gömmeleri (word embeddings) yardımcı olabilir ama maliyeti yüksek.
- Bir öğrencinin projesi adres verisinde yanlış yazılmış “mahalle” ve “sokak” ifadelerini bulmak:
  - Öğrenci regex ile “SK” ararken “WSK” (yanlış yazılmış “SK”) ve “eski” gibi kelimeler de geliyor.
  - Hoca önerisi: LCS ile “SK”, “SOK.”, “sokak” gibi doğru yazımların alt dizi olarak geçip geçmediğine bakmak.
  - PTT’nin il, ilçe, semt, mahalle ve posta kodu Excel dosyası doğrulama için kullanılabilir.
  - Sokak adlarına erişilebilirse yanlış yazılmış sokak adı edit distance ile en yakın doğru ada çevrilebilir.

Bilgiye erişim (information retrieval)

Tarihçe

- İlk örnek kütüphane veritabanları. İnternet yokken alfabetik çekmecelerdeki kartotekslerden numara bulunup kütüphaneciye verilirdi.
- Sonra metin tabanlı arama motorları geldi: Google, Yahoo, AltaVista.
  - Önce tek anahtar kelime, sonra AND ve OR ile birden çok anahtar kelime.
  - Daha sonra cümle yazınca ilgili sayfaların gelmesi.
- Çoklu ortam (multimedia) araması: IBM’in QBIC (Query by Image Content) ürünü, WebSEEk.
- Soru cevaplama (question answering) sistemleri: AnswerBus, Ask Jeeves. Türkçe sayfalar için geliştirilmiş Hakia arama motoru da vardı.

IR sistemi mimarisi



- Sonuçlar puanlanarak gelir. En ilgili belge ilk sıradadır.
- Eskiden aranan şey 2. ya da 3. sayfada bulunurdu. Bugün çoğu zaman ilk üç sonuçta bulunuyor. Sebep: iyi indeksleme ve PageRank gibi iyi puanlama algoritmaları.
- Spider’a başlangıç linkleri ve kaç seviye aşağı ineceği verilir. Sayfalardaki belgeler ve kelimeler, konumları ve adresleriyle indekslenir.
- Sayfaların çoğu dinamik. Haber sayfaları gibi sık değişen sayfalar daha sık, nadiren değişenler daha seyrek indekslenir.
- Sayfa önemini belirleyebilecek ölçütler:
  - Sayfaya kaç sayfadan link geldiği.
  - Sayfadan kaç linke gidildiği.
  - Kullanıcıların sayfada kalma süresi.
  - Sayfada sık sorgulanan kelimelerin geçmesi.
- Arama motorlarının algoritmaları gizli. Yalnızca kabaca neye bakıldığı söylenebilir.
- En önemli ölçütlerden biri sayfaya gelen link sayısı. Hocanın sayfası üniversitenin sayfasına link veriyorsa üniversitenin sayfası daha önemlidir.

IR modellerinin sınıflandırılması

- Matematiksel modeller:
  - Küme kuramsal (set-theoretic): standart Boole, genişletilmiş Boole, bulanık bilgi erişimi.
  - Cebirsel (algebraic): vektör uzayı modeli, LSA.
  - Olasılıksal (probabilistic) modeller.
- Bazı modellerde terimler (kelimeler) kullanılır, bazılarında başka özellikler.

Standart Boole modeli

- Bir terim belgede ya vardır ya yoktur.
- AND kesişim, OR birleşim işlemidir.
- Örnek: 4 terim ve 3 belge.

Terim	D1	D2	D3
Bayes' Principle	var		
Probability	var	var	var
Decision-making		var	
Bayesian Epistemology			var

- Sorgu: "probability AND decision-making".
  - S_1 = probability geçen belgeler = {D1, D2, D3}.
  - S_2 = decision-making geçen belgeler = {D2}.
  - $S_1 \cap S_2$ = {D2}.
- Avantaj: çok basit, formül gerektirmez.
- Dezavantaj: sonuçlar sırasız gelir. Terim 10 kez de geçse 1 kez de geçse aynıdır. Hoca bunu "information retrieval değil, data retrieval gibi çalışıyor" diye özetliyor.

Genişletilmiş Boole modeli

- Terimler ağırlıklandırılır. Vektör uzayı modelinden yararlanır. Sorgu ile belge arasındaki benzerlik hesaplanır ve sonuçlar sıralanır.
- Ağırlıklandırma seçenekleri: yalnızca sayım (count), terim frekansı (term frequency, TF), TF-IDF ve benzerleri.
- Slayttaki ağırlık örneği:

$$w_{x,j} = f_{x,j} \cdot \frac{\text{idf}_x}{\max_i \text{idf}_i}$$

- $f_{x,j}$: x kelimesinin j belgesindeki frekansı. $\text{idf}_x = \log \frac{N}{n_x}$: N derlemdeki (corpus) belge sayısı, n_x kelimenin geçtiği belge sayısı.
- İki terimli sorgularda benzerlik (w_1, w_2 terim ağırlıkları):

$$\text{sim}(q_{or}, d) = \sqrt{\frac{w_1^2 + w_2^2}{2}} \qquad \text{sim}(q_{and}, d) = 1 - \sqrt{\frac{(1-w_1)^2 + (1-w_2)^2}{2}}$$

- p-normu ile genelleştirme ($1 \le p \le \infty$, t terim sayısı):

$$\text{sim}(q_{or}, d) = \left(\frac{w_1^p + \cdots + w_t^p}{t}\right)^{1/p} \qquad \text{sim}(q_{and}, d) = 1 - \left(\frac{(1-w_1)^p + \cdots + (1-w_t)^p}{t}\right)^{1/p}$$

- Karma sorgu örneği: $q = (k_1 \text{ AND } k_2) \text{ OR } k_3$.
  - Önce AND kısmı hesaplanır: $a = 1 - \left(\frac{(1-w_1)^p + (1-w_2)^p}{2}\right)^{1/p}$.
  - Sonra OR: $\text{sim} = \left(\frac{a^p + w_3^p}{2}\right)^{1/p}$.
  - Her belge için hesaplanır ve belgeler benzerliğe göre sıralanır.
- Hocanın tahtadaki örneği: D1 = A A A A B, D2 = A B B, D3 = B C D E.
  - A'nın D1'deki ağırlığı: frekans 4 × A'nın IDF'si / belgedeki en büyük IDF.
  - A 3 belgenin 2'sinde geçiyor, B 3'ünde de geçiyor. A'nın IDF'si B'ninkinden büyük olduğu için payda A'nın IDF'si olur.

Vektör uzayı modeli

- Terim-belge matrisi: satırlar terimler, sütunlar belgeler. Değerler terimin belgede geçme sayısı.
- Sorgu da bir vektördür. Sorgunun her belgeyle benzerliği hesaplanır ve büyükten küçüğe sıralanır.
- TF-IDF örneği:
  - 100 kelimelik bir belgede "cow" 3 kez geçiyor: TF $= 3/100 = 0{,}03$.
  - 10 milyon belgenin 1000'inde geçiyor: IDF $= \log(10^7/10^3) = 4$.
  - TF-IDF $= 0{,}03 \times 4 = 0{,}12$.
- İki vektörün benzerliği aralarındaki açıyla ölçülür. Açıyı doğrudan hesaplamak yerine kosinüs benzerliği kullanılır:

$$\cos\theta = \frac{A \cdot B}{\|A\|\,\|B\|} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2}\,\sqrt{\sum_i b_i^2}}$$

- Örnek: $A = (1,1,1)$, $B = (1,0,1)$, $C = (10,5,9)$.
  - $\cos(A,B) = \frac{2}{\sqrt{3}\sqrt{2}} \approx 0{,}82$.
  - $\cos(A,C) = \frac{24}{\sqrt{3}\sqrt{206}} \approx 0{,}97$.
- Slayt örneği: $D_1 = 2T_1 + 3T_2 + 5T_3$, $D_2 = 3T_1 + 7T_2 + T_3$, $Q = 0T_1 + 0T_2 + 2T_3$.
  - $\cos(Q, D_1) = \frac{10}{2\sqrt{38}} \approx 0{,}81$.
  - $\cos(Q, D_2) = \frac{2}{2\sqrt{59}} \approx 0{,}13$.
  - Q, D1'e daha çok benziyor. Açı daha küçük.
- İç çarpım (inner product): yalnızca karşılıklı elemanlar çarpılıp toplanır. İkili vektörlerde ortak terim sayısını verir. Sonuç 0-1 aralığına normalize edilebilir.

İndeksleme: ters çevrilmiş dosya indeksi

- Örnek belgeler:
  - T_0 = "it is what it is"
  - T_1 = "what is it"
  - T_2 = "it is a banana"
- Sorgu "what", "is", "it":
  - "what": {0, 1}. "is": {0, 1, 2}. "it": {0, 1, 2}.
  - Kesişim: {0, 1}.
- Ters çevrilmiş indeks yalnızca belge adreslerini tutar:
  - "a": {2}, "banana": {2}, "is": {0, 1, 2}, "it": {0, 1, 2}, "what": {0, 1}.
- Tam ters çevrilmiş indeks belge ve konum tutar (konumlar 0'dan başlar):
  - "a": {(2, 2)}, "banana": {(2, 3)}.
  - "is": {(0, 1), (0, 4), (1, 1), (2, 1)}.
- Veri yapısı olarak her kelimeden bir bağlı liste çıkar. Listenin her elemanında belge numarası ve konum vardır.
- İndeksleme, her aramada belgelere tek tek bakmamak için önceden spider ile yapılır. Webdeki her sayfa indekslenmiş değildir.

Bulanık (fuzzy) mantık

- Klasik mantık: 0 ya da 1.
- Olasılık: değerlerin toplamı 1'dir.
- Bulanık mantık: bir elemanın bir kümeye üyelik derecesi vardır. Toplam 1'i geçebilir.
- Örnekler:
  - "Joe uzun": 1,57 boyundaki biri için 1,90 çok uzundur. 1,87 boyundaki biri için değil. Uzunluk göreceli.
  - Hem ekonomi hem siyaset içeren bir yazı: ekonomiye 0,8, siyasete 0,7 üyelik. Toplam 1,5 olduğu için bu olasılık olamaz.
  - "Genç" kümesi: 23 yaşındaki Charlie 1, 28 yaşındaki Ann 0,8, 35 yaşındaki Bob 0,1 üyelikle.

- Yaş grupları grafiği:
  - Genç: 0-25 yaş arası üyelik 1. 25-40 arasında azalır, 40'ta 0.
  - Orta yaş: 25-55 arası. Ortasında (40) üyelik 1.
  - Yaşlı: 40'tan itibaren artar, 55'ten sonra üyelik 1.
- Bulanık işlemler: $A = (1;\ 0{,}20;\ 0{,}75)$ ve $B = (0;\ 0{,}45;\ 0{,}50)$ için:
  - Birleşim (OR, maksimum): $(1;\ 0{,}45;\ 0{,}75)$.
  - Kesişim (AND, minimum): $(0;\ 0{,}20;\ 0{,}50)$.
  - A'nın tümleyeni (NOT, $1 - x$): $(0;\ 0{,}80;\ 0{,}25)$.

Karma min-maks modeli (MMM)

- d_{A_i}: A_i teriminin belgedeki ağırlığı.

$$\text{SIM}(Q_{or}, D) = C_{or1} \cdot \max(d_{A_1}, \dots, d_{A_n}) + C_{or2} \cdot \min(d_{A_1}, \dots, d_{A_n})$$

$$\text{SIM}(Q_{and}, D) = C_{and1} \cdot \min(d_{A_1}, \dots, d_{A_n}) + C_{and2} \cdot \max(d_{A_1}, \dots, d_{A_n})$$

- OR sorgusunda max'ın etkisi büyük olsun diye $C_{or1} > C_{or2}$. AND sorgusunda min'in etkisi büyük olsun diye $C_{and1} > C_{and2}$.
- $C_{or2} = 1 - C_{or1}$, $C_{and2} = 1 - C_{and1}$.
- Deneysel çalışmalardan çıkan değerler: $C_{or1} > 0{,}2$ ve $C_{and1} \in [0{,}5;\ 0{,}8]$.
- Bu model standart Boole modelinden çok daha iyi sonuç verir.

Ders sonu

- Hoca öğrencilere projelere başlayıp başlamadıklarını sordu. Öğrenciler başladıklarını söyledi.
- Bir sonraki ders: LSA, değerlendirme ölçütleri ve muhtemelen bilgi çıkarımı (information extraction).

Ders sonrası notlar

- Bu derste Levenshtein yerine koyma maliyeti 2 alındı. Yaygın tanımda yerine koyma maliyeti 1'dir; 2 maliyetli biçim bir çeşittir. Sonuçlar seçilen maliyete bağlıdır.
- Kosinüs benzerliği örneği: hoca "A tabii ki B'ye benzer" dedi ama hesabı tamamlamadı. Hesap tamamlanınca $\cos(A, C) \approx 0{,}97$, $\cos(A, B) \approx 0{,}82$ çıkıyor. Kosinüs benzerliği vektörlerin büyüklüğüne değil yönüne baktığı için A, C'ye daha çok benziyor.
- Bulanık kesişim örneğinde hoca önce "1; 0,20; 0,50" dedi. İlk eleman $\min(1, 0) = 0$ olmalı. Doğrusu $(0;\ 0{,}20;\ 0{,}50)$.
- Tahtadaki genişletilmiş Boole örneğinde IDF, logaritma alınmadan N/n olarak hesaplandı (A için 3/2 = 1,5, B için 3/3 = 1). Tanımda logaritma var. Logaritma ile B'nin IDF'si 0 olur.
- Hoca LCS'yi tanıtırken "en uzun alt string" diyor. LCS bir alt dizidir (subsequence), karakterler ardışık olmak zorunda değil. Ardışık olan "en uzun ortak alt dize" (longest common substring) ayrı bir problemdir.
- LCS örneğinde altyazıda Y dizesi "ATCGGG" diye geçiyor. Slaytta doğrusu "ATTCGG".
- Sonek ağacı arama karmaşıklığı altyazıda iki yerde de "O(m)" geçiyor. Uzunluğu n olan bir kalıbı aramak $O(n)$ sürer.
- Hoca bir sonraki konuyu bir yerde "LSTM" diye söylüyor. Ders sonunda bunun cebirsel model LSA olduğu anlaşılıyor.
- Altyazıdaki "Bullun/boğulun model" Boole (Boolean) modeli, "kosan simularite" kosinüs benzerliği, "Lievenstein" Levenshtein, "safiks tri" suffix tree demektir.

İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.